

# Resumen Técnico Completo: Desarrollo Android

Basado en documentación DAM2 (113 págs)

## Índice

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>1. 1. Entorno de Trabajo y Fundamentos</b>            | <b>3</b> |
| 1.1. Tipos de Aplicaciones y Entornos . . . . .          | 3        |
| 1.2. Runtime y Compilación . . . . .                     | 3        |
| 1.3. Herramientas del SDK . . . . .                      | 3        |
| <b>2. 2. Estructura y Manifiesto</b>                     | <b>3</b> |
| 2.1. El Manifiesto (AndroidManifest.xml) . . . . .       | 3        |
| 2.2. Permisos . . . . .                                  | 3        |
| <b>3. 3. Recursos y Pantallas</b>                        | <b>4</b> |
| 3.1. Recursos (res/) . . . . .                           | 4        |
| 3.2. Programando para distintas pantallas . . . . .      | 4        |
| <b>4. 4. Actividades (Activity)</b>                      | <b>4</b> |
| 4.1. Ciclo de Vida Completo . . . . .                    | 4        |
| 4.2. Guardar Estado (Cambios de configuración) . . . . . | 4        |
| <b>5. 5. Interfaz de Usuario (UI)</b>                    | <b>5</b> |
| 5.1. Layouts (Contenedores) . . . . .                    | 5        |
| 5.2. Gestión y Eventos . . . . .                         | 5        |
| 5.3. Notificaciones UI . . . . .                         | 5        |
| <b>6. 6. Intents y Comunicación</b>                      | <b>5</b> |
| 6.1. Tipos . . . . .                                     | 5        |
| 6.2. Intent Filters . . . . .                            | 5        |
| 6.3. Obtener Resultados (Activity Result API) . . . . .  | 5        |
| <b>7. 7. Broadcast Receivers</b>                         | <b>6</b> |
| <b>8. 8. Servicios (Services)</b>                        | <b>6</b> |
| <b>9. 9. Almacenamiento y Datos</b>                      | <b>6</b> |
| 9.1. SharedPreferences . . . . .                         | 6        |
| 9.2. Almacenamiento de Ficheros . . . . .                | 6        |
| 9.3. Base de Datos SQLite . . . . .                      | 6        |
| 9.4. Content Providers . . . . .                         | 7        |
| <b>10. 10. Listas y Adaptadores</b>                      | <b>7</b> |
| 10.1. Componentes . . . . .                              | 7        |
| 10.2. Optimización (ViewHolder) . . . . .                | 7        |
| <b>11. 11. Fragmentos (Fragments)</b>                    | <b>7</b> |
| 11.1. Ciclo de vida específico . . . . .                 | 7        |
| 11.2. Gestión Dinámica . . . . .                         | 7        |
| 11.3. Comunicación . . . . .                             | 7        |
| <b>12. 12. Hilos y Asincronía</b>                        | <b>8</b> |
| 12.1. AsyncTask (Legacy) . . . . .                       | 8        |

|                                  |          |
|----------------------------------|----------|
| <b>13.13. Google Maps API v2</b> | <b>8</b> |
| 13.1.Requisitos . . . . .        | 8        |
| 13.2.Uso Básico . . . . .        | 8        |
| 13.3.Localización . . . . .      | 8        |

# 1. Entorno de Trabajo y Fundamentos

## 1.1 Tipos de Aplicaciones y Entornos

- **Nativas:** Desarrolladas en Java/Kotlin con Android Studio. Máximo rendimiento y acceso al hardware.
- **Web:** HTML/CSS/JS ejecutándose en navegador. Multiplataforma pero acceso limitado.
- **Híbridas:** HTML5 en contenedor nativo (ej. Ionic, Cordova).
- **Android Studio:** IDE oficial basado en IntelliJ IDEA. Usa **Gradle** como gestor de construcción.

## 1.2 Runtime y Compilación

- **Dalvik (Antiguo):** Usaba compilación JIT (Just In Time). Traduce bytecode al ejecutar.
- **ART (Android Runtime - Moderno):** Usa compilación AOT (Ahead Of Time). Compila al instalar la app (crea archivo ELF). Mejora rendimiento y batería, pero la instalación es más lenta y ocupa más.

## 1.3 Herramientas del SDK

- **AVD Manager:** Crea dispositivos virtuales para pruebas.
- **ADB (Android Debug Bridge):** Puente de comandos PC-Dispositivo.
  - `adb devices`: Lista dispositivos.
  - `adb shell`: Abre terminal en el dispositivo.
  - `adb install app.apk`: Instala aplicaciones.
  - `adb logcat`: Muestra logs del sistema.
- **sqlite3:** Herramienta de línea de comandos para gestionar BDs en el dispositivo.

# 2. Estructura y Manifiesto

## 2.1 El Manifiesto (AndroidManifest.xml)

Archivo esencial que describe la app al sistema. Define:

- **Paquete Java:** Identificador único.
- **Componentes:** Activities, Services, Receivers, Providers.
- **Permisos:** Qué necesita la app (Internet, Cámara, etc.).
- **Versiones API:** `minSdkVersion`, `targetSdkVersion`.

## 2.2 Permisos

- **Normales:** Se conceden automáticamente al instalar (ej. Internet).
- **Peligrosos (Runtime):** Desde Android 6.0 (API 23), se deben pedir **en tiempo de ejecución** (ej. Cámara, Contactos, Ubicación).

Listing 1: Pedir permisos en tiempo de ejecución

```
if (checkSelfPermission(Manifest.permission.CAMERA) != PackageManager.PERMISSION_GRANTED) {  
    requestPermissions(new String[]{Manifest.permission.CAMERA}, CODIGO_PETICION);  
}
```

## 3 3. Recursos y Pantallas

### 3.1 Recursos (res/)

Se separan del código para facilitar traducción y adaptación.

- `drawable/`: Imágenes y XMLs gráficos.
- `layout/`: Interfaces de usuario (XML).
- `mipmap/`: Iconos de la app.
- `values/`: Cadenas (`strings.xml`), colores, estilos, dimensiones.

### 3.2 Programando para distintas pantallas

- **Unidades:** Usar **dp** (density-independent pixels) para layouts y **sp** para texto. Nunca px.
- **Calificadores:** Carpetas con sufijos para recursos alternativos.
  - `layout-land`: Para orientación horizontal.
  - `values-es`: Para idioma español.
  - `drawable-xhdpi`: Para pantallas de alta densidad.
  - `layout-sw600dp`: Para pantallas con ancho mínimo 600dp (tablets).

## 4 4. Actividades (Activity)

Representan una pantalla con interfaz. Heredan de `Activity` o `AppCompatActivity`.

### 4.1 Ciclo de Vida Completo

1. **onCreate()**: Inicialización, inflado de layout (`setContentView`).
2. **onStart()**: La actividad se hace visible.
3. **onResume()**: La actividad entra en primer plano (interactuable).
4. **onPause()**: Pierde foco parcialmente. Guardar datos persistentes aquí.
5. **onStop()**: Ya no es visible.
6. **onDestroy()**: Fin de la actividad.
7. **onRestart()**: Vuelve de `onStop` a `onStart`.

### 4.2 Guardar Estado (Cambios de configuración)

Al rotar la pantalla, la Activity se destruye y recrea. Para no perder datos temporales (variables):

Listing 2: Guardar y Restaurar Estado

```
@Override // Guardar
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putInt("contador", variableContador);
}
@Override // Recuperar (en onCreate)
protected void onCreate(Bundle savedInstanceState) {
    if (savedInstanceState != null) {
        variableContador = savedInstanceState.getInt("contador");
    }
}
```

## 5. Interfaz de Usuario (UI)

### 5.1 Layouts (Contenedores)

- **LinearLayout:** Elementos en fila única (vertical u horizontal). Usa `layout_weight` para pesos.
- **RelativeLayout:** Posicionamiento relativo ("debajo de X", "alineado con padre").
- **ConstraintLayout:** Layout plano y potente para evitar anidamientos. Usa restricciones ("constraints").
- **FrameLayout:** Para apilar vistas (ej. contener un Fragmento).

### 5.2 Gestión y Eventos

Acceso a controles mediante `findViewById(R.id.nombre)`.

Listing 3: Click Listener (Forma recomendada)

```
Button boton = findViewById(R.id.miBoton);
boton.setOnClickListener(new View.OnClickListener() {
    @Override public void onClick(View v) {
        // Código al pulsar
    }
});
```

### 5.3 Notificaciones UI

- **Toast:** Mensaje efímero.
- **Snackbar:** Barra inferior; puede tener botones de acción (ej. "Deshacer").

## 6. Intents y Comunicación

Mensajes para solicitar acciones o lanzar componentes.

### 6.1 Tipos

- **Explícitos:** Definen la clase destino exacta (navegación interna).
- **Implícitos:** Definen una acción (`ACTION_VIEW`, `ACTION_SEND`). Android busca qué app puede hacerlo.

### 6.2 Intent Filters

Declaran en el Manifiesto qué Intents implícitos puede atender una Activity.

```
<intent-filter>
    <action android:name="android.intent.action.SEND"/>
    <data android:mimeType="text/plain"/>
</intent-filter>
```

### 6.3 Obtener Resultados (Activity Result API)

Forma moderna de recibir datos de otra actividad (sustituye a `startActivityForResult`).

Listing 4: `ActivityResultLauncher`

```
// Registrar el contrato
ActivityResultLauncher<Intent> launcher = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    result -> {
        if (result.getResultCode() == RESULT_OK) {
            // Procesar data
        }
    });
// Lanzar
launcher.launch(new Intent(this, SegundaActivity.class));
```

## 7 7. Broadcast Receivers

Componentes que .escuchan"mensajes del sistema o de apps.

- **Estáticos:** Declarados en Manifiesto (limitados en versiones recientes).
- **Dinámicos:** Registrados en código (registerReceiver).

Listing 5: Ejemplo Receiver SMS

```
public class SmsReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        // Leer PDUs del SMS entrante
    }
}
```

## 8 8. Servicios (Services)

Ejecución en segundo plano sin UI.

- **Started Service:** Iniciado por startService(). Hace una tarea y debe pararse a sí mismo. Callback: onStartCommand.
- **Bound Service:** Enlazado a un cliente (bindService). Interacción cliente-servidor. Callback: onBind.

**Nota:** Por defecto corren en el hilo principal (UI Thread). Si bloquean, usar Hilos o AsyncTask.

## 9 9. Almacenamiento y Datos

### 9.1 SharedPreferences

Para configuraciones simples (clave-valor).

Listing 6: Uso de SharedPreferences

```
SharedPreferences prefs = getSharedPreferences("MisPrefs", MODE_PRIVATE);
// Escribir
prefs.edit().putString("usuario", "admin").apply();
// Leer
String user = prefs.getString("usuario", "nulo");
```

### 9.2 Almacenamiento de Ficheros

- **Interno:** Privado de la app. Se borra al desinstalar.
- **Externo:** Compartido (SD). Requiere permisos.

### 9.3 Base de Datos SQLite

BD relacional integrada. Se usa SQLiteOpenHelper para gestión.

Listing 7: Helper SQLite

```
public class MiDbHelper extends SQLiteOpenHelper {
    public MiDbHelper(Context ctx) { super(ctx, "miBD", null, 1); }
    @Override public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE usuarios (_id INTEGER PRIMARY KEY, nombre TEXT)");
    }
    @Override public void onUpgrade(SQLiteDatabase db, int old, int newVersion) {}
}
```

Listing 8: Operaciones CRUD SQLite

```
SQLiteDatabase db = helper.getWritableDatabase();
// INSERT
ContentValues v = new ContentValues();
v.put("nombre", "Ana");
db.insert("usuarios", null, v);

// QUERY
```

```
Cursor c = db.rawQuery("SELECT nombre FROM usuarios", null);
if (c.moveToNext()) {
    String nombre = c.getString(0);
}
c.close();
```

## 9.4 Content Providers

Exponen datos de una app a otras (ej. acceder a Contactos del teléfono). Se accede mediante ContentResolver y URIs.

## 10 10. Listas y Adaptadores

Para mostrar colecciones de datos (ListView, GridView, Spinner).

### 10.1 Componentes

- **AdapterView:** La vista (ListView).
- **Adapter:** El puente que convierte datos en Vistas (filas).
- **Fuente de Datos:** Array, Cursor de BD, Lista.

### 10.2 Optimización (ViewHolder)

Es crítico reciclar vistas para el rendimiento.

Listing 9: Patrón ViewHolder en getView

```
public View getView(int position, View convertView, ViewGroup parent) {
    ViewHolder holder;
    if (convertView == null) { // Si no hay vista reciclable, crear nueva
        convertView = inflater.inflate(R.layout.fila, null);
        holder = new ViewHolder();
        holder.texto = convertView.findViewById(R.id.texto);
        convertView.setTag(holder);
    } else {
        holder = (ViewHolder) convertView.getTag(); // Reciclar
    }
    holder.texto.setText(datos.get(position));
    return convertView;
}
```

## 11 11. Fragmentos (Fragments)

Sub-componentes reutilizables con su propio ciclo de vida. Siempre viven dentro de una Activity.

### 11.1 Ciclo de vida específico

onAttach → onCreate → onCreateView (inflar UI) → onActivityCreated ... onDestroyView.

### 11.2 Gestión Dinámica

Usar fragmentManager y FragmentTransaction.

Listing 10: Cambiar Fragmento

```
getSupportFragmentManager().beginTransaction()
    .replace(R.id.contenedor, new MiFragment())
    .addToBackStack(null) // Para que el botón 'Atrás' funcione
    .commit();
```

### 11.3 Comunicación

**Nunca** comunicar dos fragmentos directamente. Usar la Activity como intermediaria mediante **Interfaces (Callbacks)**. 1. Fragment define interfaz. 2. Activity implementa interfaz. 3. Fragment llama al método de la interfaz.

## 12 12. Hilos y Asincronía

El hilo principal (UI Thread) no debe bloquearse más de 5 segundos (error ANR).

### 12.1 AsyncTask (Legacy)

Clase para tareas cortas en segundo plano.

- `onPreExecute()`: En UI.
- `doInBackground()`: **En hilo secundario**.
- `onPostExecute()`: En UI (recibe resultados).

## 13 13. Google Maps API v2

### 13.1 Requisitos

1. Instalar Google Play Services en SDK. 2. Obtener **API Key** en consola Google Cloud (usando SHA-1 del certificado). 3. Permisos: `INTERNET`, `ACCESS_FINE_LOCATION`, `GSERVICES`. 4. Añadir meta-data con la Key en Manifiesto.

### 13.2 Uso Básico

Usar `MapFragment` o `MapView`.

Listing 11: Configurar Mapa

```
// En onMapReady
GoogleMap mapa = ...;
LatLng punto = new LatLng(42.38, -8.71);

// Marcador
mapa.addMarker(new MarkerOptions().position(punto).title("IES Chan do Monte"));

// Cámara
mapa.moveCamera(CameraUpdateFactory.newLatLngZoom(punto, 15));

// Tipos de mapa
mapa.setMapType(GoogleMap.MAP_TYPE_HYBRID);
```

### 13.3 Localización

Uso de `LocationManager` para obtener coordenadas GPS/Red. Requiere gestionar permisos en tiempo de ejecución y usar `LocationListener` para actualizaciones (`onLocationChanged`).